Unauthenticated Access and Memory Exhaustion in Zebra's tracing /filter Endpoint

Summary

For operational convenience, Zebra can be configured with an externally accessible /filter web endpoint. Once exposed, the endpoint requires no privileges. Under the affected configuration, an attacker only needs to upload a large request to make zebrad's memory usage grow rapidly. In testing, a 38 MB request increased process memory to approximately 1,981 MB (nearly 2 GB). After memory is exhausted, the operating system kills zebrad and, in severe cases, may also affect other services on the same host. The main risk is remote node termination and potentially destabilizing the entire host.

The same endpoint also allows unauthenticated users to read or change logging levels. An attacker can first enable the most verbose logging to increase CPU, disk, and memory pressure, and then launch the large-request attack. The issue remains exploitable after the node restarts, so the impact can become a sustained denial of service rather than a one-time slowdown.

This endpoint is not enabled by default. The attack surface exists only when zebrad is compiled with the filter-reload feature and tracing.endpoint_addr is configured. When both conditions are met, the latest stable version, zebrad v6.3.0, remains affected. This report reproduced the issue in an isolated Regtest environment.

Mitigating Factors:

- Default installations are generally unaffected. If filter-reload is not enabled and tracing.endpoint_addr is not configured, the attackable /filter endpoint does not exist.
- Even when the feature is enabled, an external attacker cannot connect directly if the endpoint listens only on a local address such as 127.0.0.1.
- The observed consequences are primarily node or host unavailability. There is no evidence that an attacker can use this issue to steal funds, read confidential data, or alter blockchain consensus.
- The operating system normally records "Out of memory" and the name of the killed process in dmesg, allowing administrators to distinguish memory exhaustion from an ordinary crash.

Aggravating Factors:

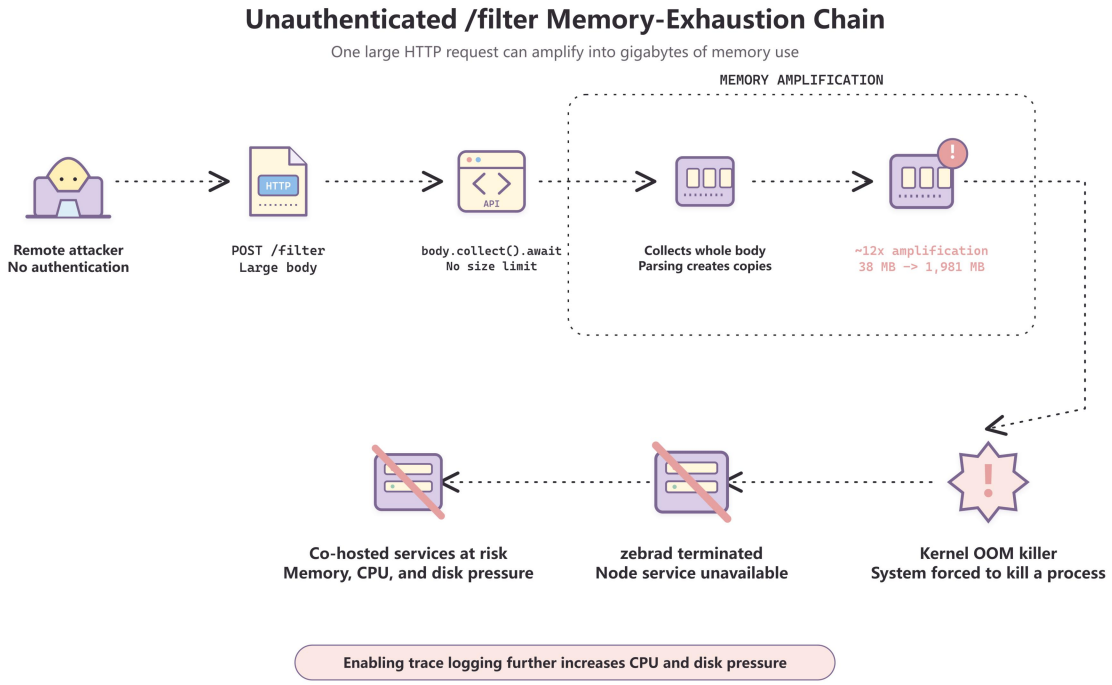
- The endpoint requires no login or token. Anyone who can reach it over the network can read or change logging settings and launch the attack.
- There is no request-size limit, and the request produces approximately 12x memory amplification. Sending tens or hundreds of megabytes may consume several gigabytes of memory.
- When memory is exhausted, the operating system decides which process to kill. The victim may not be limited to zebrad; databases, wallets, or monitoring services on the same host may also be terminated, and the worst case can destabilize the entire machine.
- The attack uses an ordinary HTTP POST request, is simple and inexpensive to construct, and requires neither knowledge of the Zcash protocol nor mining power.

- Restarting does not remove the vulnerability. As long as the endpoint remains externally accessible, an attacker can repeatedly send large requests and prevent the node from operating reliably for an extended period.

Vulnerability Details:

The vulnerable endpoint is read_filter() at zebrad/src/components/tracing/endpoint.rs:37. It calls .collect().await directly on the body of POST /filter requests, with no size limit and no Content-Length pre-check. After the entire body is collected in memory, parsing it as a filter string further increases memory usage. Testing measured approximately 12x amplification.

The attack call chain is shown below:



Unauthenticated writes to the same endpoint are an independent security issue. An attacker can remotely execute POST /filter, for example zebrad=trace,zebra_network=trace, to modify the logging filter arbitrarily. Trace-level logging substantially increases CPU and I/O consumption and dilutes audit logs, so it can be used as a preparatory amplification step for the OOM attack.

Technical Details:

Trigger Conditions

- zebrad is compiled with --features filter-reload, which is not a default feature.
- tracing.endpoint_addr is configured and bound to an address reachable by the attacker, such as 0.0.0.0:3000.
- The attacker can reach the port over the network; no credentials are required.

Dynamic Evidence

Latest-version test environment: zebrad v6.3.0, release build with filter-reload, isolated Regtest, two-host setup (Windows attacker -> Linux victim VM):

```
[1] GET /filter  -> HTTP 200; current filter read successfully without
credentials
[2] POST /filter -> filter changed successfully without credentials (victim log
shows reloading tracing filter)
[3] Large body upload -> victim RSS 38 MB -> 1,981 MB -> connection interrupted
[4] Connection to the port refused -> process is dead
[5] Victim dmesg: Out of memory: Killed process ... (zebrad) anon-rss:...
```

Security Impact

- A single request can cause the operating system's OOM killer to terminate the node process.
- The issue can be triggered again after the process restarts.
- Other processes on the same host may also be killed, extending the impact beyond the node itself.
- No information disclosure, financial loss, or consensus fork has been demonstrated.
- The primary impact is complete loss of availability.

Long-Term Remediation Recommendations

- Add a hard request-body size limit to /filter: first reject oversized requests using a Content-Length pre-check, then enforce a streaming limit while reading, such as take(limit), and return HTTP 413 when the limit is exceeded.
- Add authentication and authorization to the endpoint, such as the cookie authentication used by RPC, or bind it to loopback by default.
- Apply a length limit and validity checks to the filter string itself, and reject oversized values immediately.
- Deployment mitigation: use a cgroup limit, for example systemd-run -p MemoryMax=..., to cap zebrad memory and reduce the OOM blast radius.
- Add regression tests verifying that an oversized body is rejected while the process remains alive, unauthenticated requests are rejected, and valid filter changes continue to work.